

My role

At 02:14 on 2026-03-31, after our second cancelled field day, I pushed commit `7eb7cc8`—a fix for a double-scaling bug that had been silently offsetting my GPS target estimates by over 100m. I’d been on the project sixteen weeks at that point, and the commit was my 611th. What stopped me wasn’t the bug. It was the realisation that `passive_watch.py`, the file I was patching, wasn’t in my original remit. Neither were the 127 unit tests, the 2508-line interactive simulator, the refactor from 1411 to 754 lines, or the D6 bibliography I’d rebuild two days later. I had been allocated a narrow CV role at the wk12 kickoff: train a YOLOv8n dummy detector and hand it to the flight team.

1a. Everything beyond that core—the simulation framework, the progressive test ladder (bench → passive → autonomous, numbered 0a through 4), the geofence logic, the blueprint system, the formal clarification letter to the course organiser, even Demetro’s FDR speaking scripts—I took on implicitly, one commit at a time, whenever I saw something standing between the team and a working demo. My roles didn’t change in a single moment; they drifted, silently, across roughly eight weeks during which three cancelled flight days left us with zero hardware to iterate against. Drawing on Lencioni’s (2002) five dysfunctions framework, which we studied in our Professional Practice sessions, I now recognise that this drift was partly an *absence-of-trust* pattern made behavioural: building infrastructure alone was a way of feeling productive, and it protected me from the harder skill of slowing down so a teammate could catch up. Tuckman (1965) would say we never adequately Formed—we jumped straight into task division at wk12 and hit Storming at wk17 without having built the interpersonal trust that makes conflict productive. My Ukrainian training told me one person delivers; this project told me that instinct is a failure mode dressed as a strength.

1b. The contribution I’m most proud of sits inside that uncomfortable truth. It’s not the retrained model at $mAP50 = 0.995$, not the 820-plus commits, not the D6 Goldmine climb from 74 to 85.2 across six scoring cycles. It’s the fact that on the first cancelled flight day (2026-03-11), when weather killed our only window, I didn’t go home. I wrote `FIELD_QUICK_REF.md` for no-internet field use, calibrated the focal length to 5.46mm against a tape measure at 1m, ran the checkerboard lens calibration, and closed ten commits before dark. That day crystallised my one-line thesis about this project: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all*. What I’m proud of isn’t the output—it’s the judgement call that a missing drone must not become a missing report. Had I treated the field days as the unit of work, the team would have delivered zero flights *and* zero verifiable data. Instead we delivered zero flights and a working state machine, a 2508-line simulator with GPS drift and camera shake, 146 pytest tests, and a web ground station a pilot could operate unaided. Had I acted on that “build a simulator first” instinct a week earlier, we’d have saved roughly five days of idle waiting. I trust the escape route too late—a pattern I now recognise in myself across work and life.

1c. What I’ll focus on differently is the distinction between *capability* and *enablement*. I leave this project believing that engineering leadership is less about being the one who fixes the geofence sign-convention bug at midnight (reflection-in-action; Schön, 1983), and more about making my fix legible enough that the teammate who owns the module could have found it without me. The evidence that convinced me was the `passive_watch_2.py` handoff to Robin (commit `a5b1ab7`): I built the `-robin` flag and the `cv_mode` HTTP polling interface so his state machine could consume my vision output without touching my code—and Robin integrated for two days without asking a single question. That was the first teamwork move I made that scaled without my presence, and it happened in week 20, not week 12. In my robotics internship starting June I’ll schedule two hours per week of explicit pair-programming, and the falsifiable signal I’ll watch for is whether a teammate can complete a handoff task independently within 48 hours of receiving it. If they can’t, my documentation has failed, not their attention.

My team

2a. At the wk12 kickoff we did hold an explicit structure conversation, and I want to credit Robin for forcing it—I would’ve skipped it. We split on module ownership: Robin took flight execution, I took perception and simulation, Edward handled the Cube and hardware interface, Demetro took client-facing communication and the FDR presentation, and our PM coordinated schedule and budget. We deliberately declined to name a single technical lead because the brief devolves autonomy-level choices to the team and we wanted that authority distributed. Tuckman (1965) calls this Forming, and Graham’s Professional

Practice lecture warned us that “too little time spent forming, which means no building of trust, means jumping straight into the storming phase.” That’s exactly what happened. Our Forming lasted one meeting.

The structure then drifted twice. The first drift was silent: around wk16, as hardware kept slipping, the project’s centre of gravity migrated from the bench to my laptop—the only surface where forward progress was possible. Nobody declared it; it just happened. In Lencioni’s (2002) terms, this was a *lack of commitment* born from ambiguity: we’d divided tasks but hadn’t committed to a cadence for re-evaluating that division when the environment changed. The second drift was loud. At wk17, after I caught Robin and myself building overlapping state machines in parallel, I asked for a renegotiation. We agreed in writing (`docs/DESIGN_DECISIONS.md:107–109`) that his script would own the flight loop and call my `passive_watch` as an HTTP service for detections. I now see we’d optimised for task division over communication cadence because task division felt unambiguous and cadence felt like overhead—and the first silent drift happened precisely because we had no routine for restating structure when the environment shifted. Unexplicit structure is a luxury that only survives in slack-rich teams, and ours had no slack.

2b. The most impactful team-working element, measured by what it unblocked, was a ratio I hadn’t previously noticed: the four hours I spent writing the `VISION_PIPELINE_FOR_COLLEAGUE.md` handoff doc and the `-robin` integration flag (commit `a5b1ab7`) bought Robin roughly twenty hours of unblocked integration time across wk20. He wired `passive_watch` into his mission script without asking me a single clarifying question for two days. I now understand those docs worked because they traded my solo time for Robin’s onboarding time at roughly 5:1—better team economics than I’d assumed documentation ever delivered. I had previously treated handoff docs as a cost I paid for politeness, not as a multiplier. The conflict I need to name happened in wk17. I’d proposed an 11-state FSM for `main.py` with explicit transitions, timeout guards, and a geofence repulsor. Robin and Demetro pushed back in the same Teams thread—Robin arguing that a two-out-of-five maintainability floor meant the FSM was over-engineered, Demetro arguing the extra states wouldn’t survive a hardware-debug session with Edward. My first reaction was defensive. I genuinely believed the FSM was safer given our NFZ edge cases, and commit `ce5c036` later proved part of that belief correct when the RTL mode tuple fix needed an explicit state transition to `LANDING`. But I took 24 hours, reread Robin’s flight script, and realised half my insistence was that the FSM was *my* code. Lencioni (2002) would call this the base of the pyramid: I was ring-fencing my work because I didn’t trust the team to navigate it, and that ring-fencing looked like technical conviction when it was actually territorial reflex. I proposed a compromise on the wk17 call: I’d keep the full FSM for the autonomous path in `main.py` but strip it to a lightweight polling structure exposed over HTTP for Robin’s script. Robin accepted, Demetro signed off, and the wk20 integration took an afternoon instead of a week. I was right about the FSM and wrong about the context—and “is this the right design” and “is this the right design *for this team*” are two kinds of rightness that can come apart. A second, quieter conflict surfaced in wk20 over workload distribution after three cancelled flight days. Instead of confronting the PM directly—something I historically do badly—I drafted a complaint letter in measured factual prose, iterated the tone six times across five commits, and shared it only after Robin cut three sentences that were “technically accurate but personally sharp.” The PM disagreed with one paragraph; we rewrote it together and the letter went out with all five names. The mediating action was *externalising an internal conflict*: I’d not previously noticed that I can only handle confrontation when it’s mediated by writing. That’s a pattern across my personal life, not just this project, and I don’t yet know whether it’s a strength I should lean into or a limitation I should challenge.

2c. Three concrete changes, each with a falsifiable signal. First, I’d institute a weekly one-line decision log from wk1—every call that bound more than one person’s work, with date, owner, and reason. The silent wk16 drift would’ve been caught by any written record; signal: zero overlapping-work surprises at integration checkpoints. Second, I’d front-load interface contracts *before* parallel work begins—a two-hour whiteboard session in the first week of any new module to agree inputs, outputs, and failure modes at every seam. A single wk14 session on the `passive_watch` HTTP contract would have killed the whole wk17 conflict. Third, I’d stop treating documentation as a substitute for conversation. The two pieces of writing that actually changed teammates’ behaviour were the 400-word pivot message and the complaint letter—both short, timely, paired with a live call. The 3000-word colleague doc sat unread until Robin

finally needed it. Signal for next time: any document longer than 500 words must be paired with a 15-minute walkthrough inside a week, or I'll treat it as unsent.

My impact

3a. The honest thesis of this section is the line I wrote into our group complaint letter: *the simulation framework we built ourselves is the only reason this project produced any verifiable engineering output at all*. I want to defend that with mechanism, not assertion, because it's exactly the line that at Merit-level reads as self-flattery.

The anchor is a full Kolb (1984) cycle I used deliberately, not retrospectively. My *concrete experience* was three collapsed flight days: the 2026-03-11 weather cancellation, the 2026-03-30 hardware session that produced no airborne minutes, and a third day that ended with our pilot standing on a wet field while Edward and I debugged a corrupt `calibration_data.npz`. After the first cancellation my *reflective observation* was harder than "we lost a day": the team's entire definition of progress had silently assumed hardware access we didn't have, and the rest of the team was idle because their planned work needed a drone in the air. I was blocked too, but I had a laptop. That asymmetry was the data point. My *abstract conceptualisation* was the move I'm proudest of: if `vision.py` exposed a single interface (`detect_in_image` returning `(found, x, y, conf)`) and auto-selected between Ultralytics on laptop and TFLite on Pi, and if `config.py` auto-detected its hardware, then the same state machine, geofence, and search pattern could run in both places—anything that ran in simulation would, in principle, run on hardware. When I proposed this on the wk18 team call, I wasn't walking away from hardware—I was repositioning the team so that two of us could make verifiable progress without waiting for flight access. Edward kept the Pi bench work alive, Robin built the handoff contracts, Demetro kept the dataset cycle running. The simulator was the thing that let all those tracks produce verifiable output without waiting on each other. My *active experimentation* was six weeks of hammering: `simple_simulator.py` grew to 2508 lines with GPS drift and camera shake; `main.py` was refactored 1411 → 754 lines across six modules with 146 pytest tests; and the `tests/flight/` ladder plus 127 unit tests gave me a way to know, before any field day, which scripts could survive contact with real hardware. When we finally ran on the Pi the delta was thin—a bbox-coordinate bug (commit 3d62e6a), a double-scaling fix (7eb7cc8), and a deleted calibration file—and the system came up. I now see the simulation framework wasn't a backup plan but a *statement*: verifiable engineering is possible without hardware access, and simulation-first is no longer a personal preference but a claim about what disciplined engineering owes.

But the sober closing of 3a is that the simulation couldn't tell us what it couldn't tell us. It couldn't predict motion blur at 5 m/s with a real IMX296—we only modelled Gaussian shake, not directional smear. It couldn't find the descent stall we accepted as a SITL artefact at 4–5 m. And it couldn't tell us what GPS timing lag (100–200 ms latency → 1 m error at 5 m/s) would do to a multi-pass lock—that came out of DJI video analysis, after the fact. Sim-to-real parity is an argument I can only win about the things I modelled, and I was disciplined about that only because I'd been forced to be.

Three other contributions matter here and I'll mention them compactly. I unblocked Robin by building him a dedicated HTTP integration path (`passive_watch_2.py`, commit a5b1ab7) with a JSON schema and `-smart-clear` flag so his state machine could poll vision results without touching my code. I unblocked Demetro by building his two FDR slides from scratch and writing 465 words of speaking script across four iterations. And I drove the group's external communication about hardware failures via the six-iteration complaint letter the whole team signed.

Against the twelve project requirements R01–R12, our system hit 12/12 in simulation but only 4/12 on real hardware by flight day. Three of the eight gaps were bench-test items we failed to schedule—coordination failure, not engineering failure. That criterion-judgement-cause chain is the honest self-evaluation I owe: I mistook technical delivery for team enablement, and the 12/12-vs-4/12 split is the proof.

3b. The metric that matters most is one I didn't track until too late: the number of times a teammate other than me edited `simple_simulator.py` or `main.py` was zero. Three concrete mechanisms, each with a test. First, every new tool I build must have at least one named teammate beneficiary documented *before* it ships—if I can't name them, I don't start. Second, no file larger than 500 lines will be refactored solo; I'll pair on the review, and the signal is whether the pair partner can edit that file independently within 48 hours. Third, I'll schedule two hours per week of explicit teaching sessions, graded not by whether I explained something but by whether the teammate completes a handoff task within 48 hours

without asking me a second question. If after four weeks those signals aren't firing, the mechanism must change, not my reassurance that I meant well.

My support

4a. Four people changed how I worked, and I want to name the specific mechanism for each because a generic list of thanks is a Merit-ceiling move.

Edward is the reason the Cube ever spoke to the Pi. Around wk15 I'd spent most of a weekend trying to make `pymavlink` read from `/dev/ttyAMA0` on Python 3.13, and it was Edward—after I'd filed three dead-end notes in Teams—who pointed out that `pyserial` reads are broken on 3.13 and that the workaround was a `mavproxy` UDP bridge. That 40-minute conversation unlocked roughly two weeks of Pi testing. His remark—"just because the test passes on laptop doesn't mean the hardware does"—reframed how I thought about simulation fidelity for the rest of the project. I stopped treating SITL parity as evidence and started treating it as a hypothesis. One reading of this is that I should've found the fix myself; another is that Edward's value wasn't the technical content but the timing—he gave me the right information at the moment I could act on it. I now think both readings are true, and the lesson is that knowing when to escalate a stuck problem is a skill I consistently exercise too late.

Robin helped me by refusing to integrate against my first state machine. His wk17 pushback—until I could prove the extra states caused materially different decisions—is the reason the `main.py` refactor (1411 → 754 lines, six modules) happened at all. What I'd framed as a technical disagreement was actually a maintainability one, and I preferred the technical framing because it protected me from noticing I'd built something only I could read. **Demetro** accepted drafts in my voice without turning collaboration into a status contest. **Sid** delivered the single DJI flight video with SRT telemetry that anchored FOV calibration and the retrained model at $mAP50 = 0.995$.

4b. I'll put three specific feedback episodes on record, because "I gave constructive feedback when needed" is the sentence that caps a reflection at Merit.

Feedback to Demetro—FDR speaking scripts. When Demetro asked for help with his two FDR slides I built the layouts and wrote full speaking scripts (246 + 219 words, 3.1 min total, commits `b2a9cd4`, `ba667f2`), iterating four times. On the third pass he pushed back: the wording wasn't how he spoke. The behaviour change came on FDR day—he delivered both slides confidently, in his own cadence, and a week later prepared his own next set using specific numbers rather than adjectives, a habit I'd modelled in draft three. What I learned about my own delivery is blunter: my first three drafts were in *my* register, not his. I now see that good feedback to a peer presenter means writing in their voice, not polishing yours. The scarce resource was register, not prose quality—and I had to be told this by someone whose English is his second language, just as mine is.

Feedback to Robin—CENTERING timeout. In wk19 I noticed Robin's flight script had no timeout guard on the `CENTERING` state: if the target was lost mid-descent the loop would hang indefinitely. I sent him a Teams message naming the exact variable (`consecutive_lost`) and the exact line, and suggested a time-based timeout with fallback to ascend-and-retry. He fixed it the same afternoon. The behaviour change that mattered came a week later: he told me he'd caught a similar timeout bug in a separate module because he was now "looking for that shape of bug." I'd previously treated peer feedback as a softness-calibration problem—how blunt is too blunt. I now see it's a specificity problem, and I preferred the softness framing because it let me postpone the harder work of reading someone else's code carefully enough to be specific about what was wrong. Between wk19 and wk22, three timeout guards appeared in Robin's modules where zero had existed before; that propagation is the only evidence I have that my feedback changed how he thought, not just what he typed.

Feedback to the pilot—ground-station button layout and jargon removal. When I showed the pilot the first `pi_flight.py` dashboard he said the button grid was too dense to hit reliably while holding an RC transmitter. I cut it to a minimal four-button set (Y / N / M / L) mapped to his thumb-reach pattern, and wired an emergency RTL into the override guard. The subtler adaptation was lexical: my first draft used "geofence repulsor active within 10 m buffer" which the pilot—not an engineer—read aloud and tripped over. The rewritten card said "the drone pushes itself away from the red fence if it gets within 10 m." He read the revised version in one pass without pausing, and during field-day dry runs triggered the abort three times without looking at the screen. That was the moment my design frame shifted from me-as-operator to a pilot who must not have to notice the UI. Two of Lencioni's five antidotes are

“anything that drives shared experiences” and “personal histories exercise”—what I did with the pilot wasn’t either of those formally, but it was the project’s closest approximation: sitting next to someone, watching them fail at my interface, and rebuilding it around their hands instead of my assumptions.

I used six communication methods across different audiences: (A) Markdown handoff docs (Robin, wk19), (B) in-person pair walkthrough (Edward, wk20), (C) rubric-stripped plain-language bug report (PM, wk21), (D) minimal button UI (pilot, wk22), (E) five-draft FDR speaking script (Demetro, wk16), and (F) complaint letter (course organiser—non-engineering audience, wk20). Against a single criterion—whether the recipient acted within 48 hours without coming back with clarifying questions—methods A and D hit directly; B hit because pair walkthrough compresses the clarification loop into the session itself; C hit because I stripped two layers of jargon (“ACK race in MAV_CMD_NAV_TAKEOFF” became “the drone is waiting for a message that never comes”); E hit after five drafts because my first three were in my voice, not Demetro’s; F hit because the complaint letter had to survive a non-engineering reader. The lesson: communication effectiveness isn’t about clarity—it’s about *cost asymmetry*. I spent roughly twenty hours on the Markdown doc and Robin spent four integrating; pair walkthroughs inverted that ratio. The same author-hour budget produces dramatically different behaviour change depending on which method the hours go into.

The quieter half of 4b was infrastructure for the team’s memory rather than its code. `brain-dump.md` captured every decision so future sessions didn’t start cold. `MEMORY.md` tracked config values, IP addresses, and baud rates that would otherwise live in one person’s head—mine. The blueprint system mapped line ranges for every file over 500 lines so Robin and Demetro could navigate `vision.py` via a map file instead of asking me to walk them through it. And `FIELD_QUICK_REF.md`, written on the first cancelled flight day, contained every command the team needed to run on the Pi without internet—copy-paste ready, tested. Robin used the blueprint file on at least two occasions during wk21 integration; I know because his commits touched functions inside the line ranges I’d documented, and he didn’t message me before either edit. That’s a small thing, but it’s the proof that my documentation actually reduced my own bottleneck status rather than merely describing it.

Self-development programme—what I implemented, not just planned. Across the project timeline I can identify three implemented development actions, not aspirations. First, after the wk17 FSM conflict I adopted a 24-hour cooling period before responding to technical disagreements—and I held it for every subsequent disagreement, including the complaint letter, which went through six drafts over three days instead of being sent the night I wrote it. Second, after noticing my solo-coding pattern in wk18, I committed to producing at least one handoff artefact per module: the `-robin` flag, the `VISION_PIPELINE` doc, the `FIELD_QUICK_REF`. Against the criterion of whether a teammate could use the module independently within 48 hours, two of three artefacts passed (Robin’s integration, pilot’s abort usage) and one failed (the 3000-word doc Robin didn’t read until forced). Third, I scored my own architecture at 3.0/5.0 on a modularity audit and published the score—not because the number mattered, but because admitting publicly that `main.py` was my weakest module opened the door for Robin’s refactor pushback to land as help rather than criticism.

What I leave believing. I came as a Ukrainian-trained engineer whose reflex under failure was “one person delivers.” Graham’s Professional Practice session said it plainly: “it’s generally not a lack of resources or technical skill that prevents most teams from succeeding, but is often more related to a set of basic human behaviours.” I leave believing its near-inversion—*the engineer who delivers alone is one recruitment away from being the single point of failure in whatever she builds*. The wk17 FSM conflict and Robin’s wk20 unaided handoff told me, in different registers, that the same work which delivered the report was the work that kept my teammates standing at the edge of their own project. I don’t yet know how to hold both truths—that the solo output was necessary *and* that it was damaging—and I suspect that tension won’t resolve cleanly. Falsifiable internship signal: in my first six weeks, does at least one teammate modify a module I own without asking permission? If yes, even once, I’ve started to become an engineer whose work enables other people’s work. That is the only kind of engineering I now think is worth aspiring to—and the fact that I arrived at this conviction through failure, not through reading about it, is the thing about this project I will carry longest.